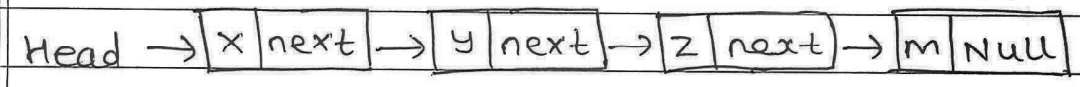
Name :- JayKishan J Saharan
Div :- D Roll no. :- 26
courselProgram :- BTech CSE SY
Subject :- DSA

DSA lab assignment 8.

Q.1) what is a Singly Linked List? Draw the node structure and represent it using class?

Sol^n :- Singly Linked List is the simplest form of Linked list in which the node contains two members data and a next pointer that stores a address of the next node.



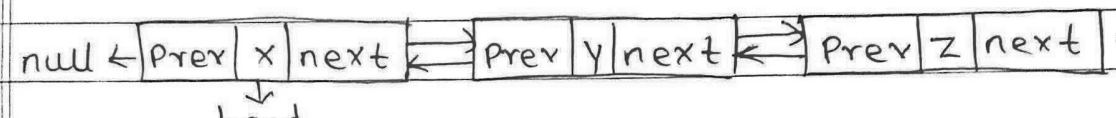
- each node in a singly Linked list is connected through to next pointer and the last node to the null denoting the end of the Linked List.

=>
```
class Node {
Public:
 int data;
 Node * next;
 Node (int value) {
  data = value;
  next = nullptr;
 }
};
```

```
class LinkedList {
 Private:
  Node * head;
 Public:
  LinkedList() {
   head = nullptr;
  }
};
```

Q.2) What is Doubly Linked List? Draw the code structure and represent it using class.

Sol^n:- The Doubly Linked List is the modified version of singly linked list each node of the DLL consists of three data members data, next, prev. Previous is a pointer which points to the previous node and next points the last node to null.



- Each node in the DLL except the first and last node is connected with each other through the prev and next pointer.

⇒

```
class Node {
Public:
 int data;
 Node *next, *Prev;
 Node (intvalue){
   data=value;
   next=prev=nullptr;
 }
};
```

```
class DoublyLinkedlist {
Private:
  Node * head;
Public:
 DoublyLinkedlist() {
   head=nullptr;
 }
};
```

Q.3) write an algorithm to traverse an singly Linked list.

Sol^n:

Step:1) If head == Null, Print "Empty" and exit.

Step:2) Set temp=head.

Step:3) Print "Linked List"

Step: 4). while temp != NULL:
a)- Print temp → data followed by "→"
b)- Move temp = temp → next
Step: 5) Print "NULL" to indicate end of list.

Q.4) write an algorithm to traverse an Doubly linked list in reverse.
Sol^n :-
Step: 1). If head == null, Print "Empty" and exit.
Step: 2) Initialize a empty stack.
: 3) Set temp = head
Step: 4) while temp != NULL:
a)- Push (temp → data)
b)- temp = temp → next
Step: 5) Pop element from stack.
Step: 6) Print "NULL" to indicate end of list.

Q. 5) write an algorithm to count no. of nodes in Singly LL.
Sol^n :-
Step: 1) If head == null, Print "Empty" and exit.
: 2) Set temp = head
Step: 3) Initialize count = 0
Step: 4) while temp != null:
a)- count = count + 1
b)- temp = temp → next
Step: 5) Print "Total nodes:", count.

q.6) Write an algorithm to search an element in DLL.

Sol^n :-

Step:1) Start

Step:2) check if the list is empty :
- If head == NULL,
  → Print "List is empty"
  → Exit the function.

Step:3) Initialize pointers and position counter :
- Set temp = head
- Set pos = 1

Step:4) Traverse the list :
- while temp != NULL, do
  → If temp → data == val, then
    Print "Element found", pos
    Exit the function
  → Else
    Move the next node, temp = temp → next
    Increment position counter, Pos ++

Step:5) If traversal completes without finding a element :
- Print "Element not found"

Step:6) Stop.

q.7) What are the real world applications of linked list.

Sol^n :- Applications of linked lists are:

1) Navigation in Browser :-
used to implement forward and backward navigation.

2) Music or Video playlists :
Moving to next or previous songs/videos in a playlist.

3) Undo/Redo functionality in Software:
used for undo and redo operations in text editors etc.

4) Implementation of Double-Ended Queue:
DLL's Supports insertion at both the ends.

5) Navigation in Games:
Used to manage player movements, game states or objects.

Q.8) Write algorithm/Pseudo code for insertion at end of SLL.

Sol^n :-

Step :- 1) Allocate memory for newNode.

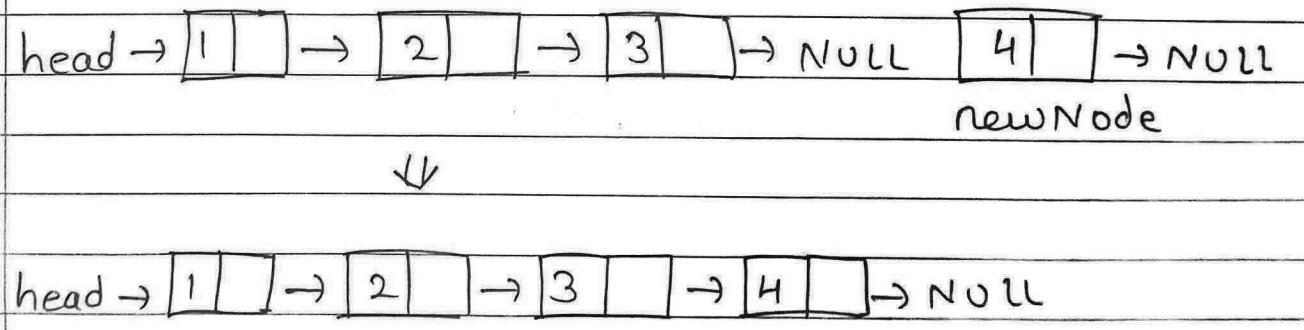
Step: 2) If head == NULL
a)- Set head = newNode
b)- Exit

Step: 3) Initialize temp = head.

Step: 4) Traverse list until temp → next == NULL using
while (temp → next != nullptr)
temp = temp → next

Step: 5) Stet temp → next = newNode

Step: 6) Print "inserted", value



Q.9) write algorithm for deletion from beginning of DLL

Sol^n :-

Step:1) check if List is empty:
- If head==NULL
  → Print "List is empty"
  → Exit

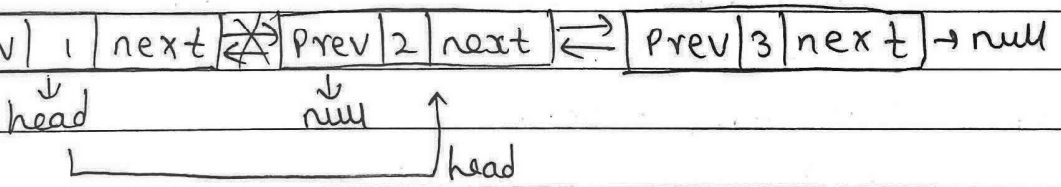
Step:2) Store the current head node
- Set temp = head

Step:3) Move the head pointer to the next node
- Set head = head → next

Step:4) check if the new head is not NULL:
- If head != NULL
  → set head → prev = NULL

Step:5) Display the deleted element
- Print "Deleted", temp → data

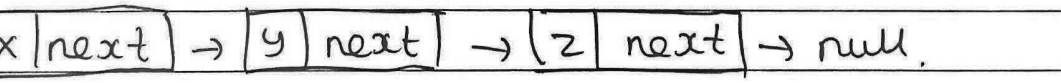
Step:6) free memory of the deleted node
- delete temp



Q.10) what are different types of linked lists.

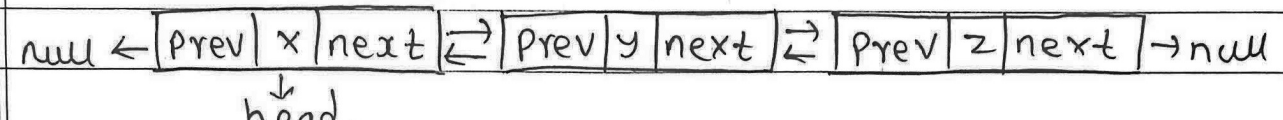
Sol^n : There are four types of linked lists.

1) Singly Linked List :-
Node contains two data members (data, next) where pointer stores the address of next data node.



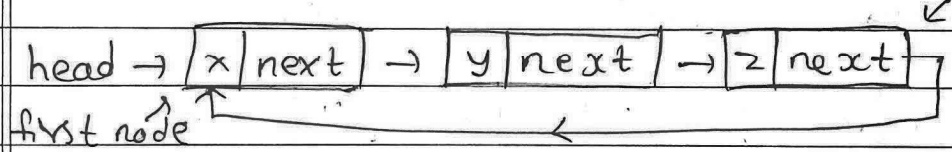
2) Doubly Linked List:

Node contains three data members (prev, data, next) where prev points to the previous node and last next node points to null and every node is connected to each other.



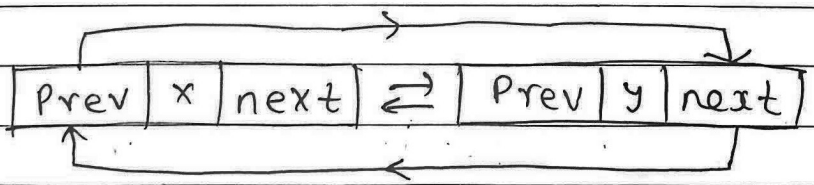
3). Circular Singly Linked List:

Node contains two data members (data, next) and the next pointer of the last node points back to the first node and forms a circle.



4) Circular Doubly Linked List.

- Each node has two pointers prev and next where the prev pointer points to the previous node and the next points to the next node and the last node (next) stores the address of first node and the first node (prev) stores the address of last node.



Q.11) write algorithm for Insertion at specific position in SLL

Sol^n :-

Step:1) If Pos < 0, Print "Invalid Position" and return.

Step:2) If Pos == 1, call function and return.

Step:3) Allocate memory for a new node.

Step:4) Initialize a pointer temp = head

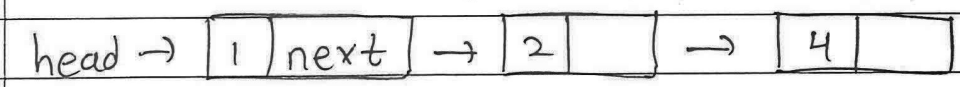
Step:5) Repeat loop for (i=1; i<POS-1 && temp != NULL; i++)
temp = temp → next

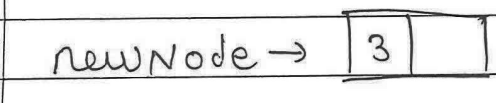
Step:6) if temp == NULL
Print "Position out or range"
Delete newNode and return

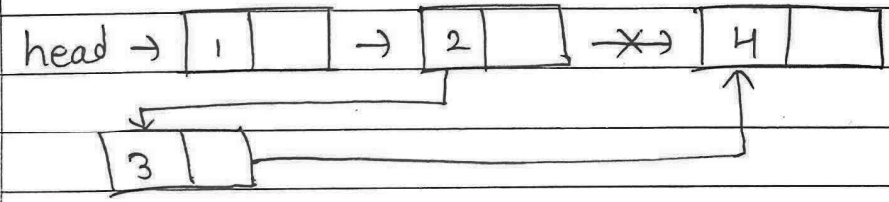
Step:7) Set newNode → next = temp → next.

Step:8) Set temp → next = newNode

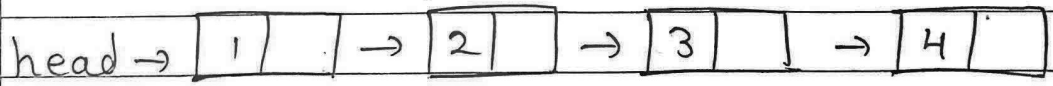
Step:9) Print "Inserted", value, "at", POS







=>



Q.12) write algorithm for insertion at specific position in DLL

Sol^n:-

Step:1) If Pos <= 0 :
→ Display "Invalid"
→ Exit

Step:2) If Pos = 1
→ call insertatBeginning(Val)
→ Exit

Step:3) Initialize a temporary pointer temp = head

:4) Traverse the list to reach node just before the desired position.
→ for i = 1 to Pos = 2, move temp = temp → next
→ If temp = NULL before reaching position display "Position out of Range" and Exit.

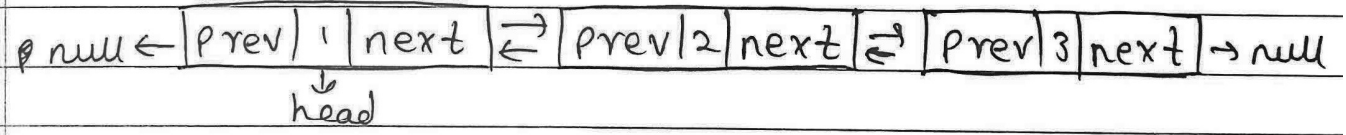
Step:5) Create a new node newNode

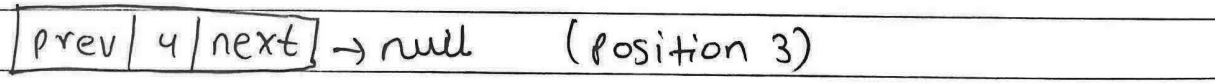
Step:6) Set
→ newNode → data = val
→ newNode → next = temp → next
→ newNode → Prev = temp

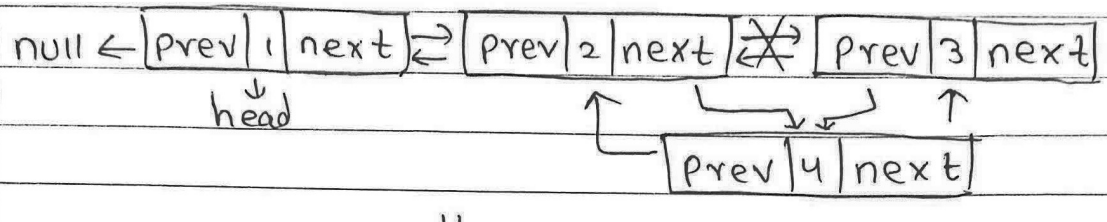
Step:7) If temp → next != NULL :
→ set temp → next → Prev = newNode

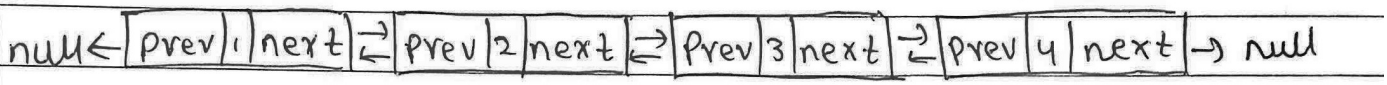
Step:8) Link the new node with the previous node
→ Set temp → next = newNode

Step:9) Display message "Inserted", Val









* Conclusion:

In this program we have successfully implemented Singly and Doubly linked list using c++. this program demonstrates various operations such as insertion, deletion, traversal (forward and backward) searching and counting number of nodes.